



HEKATE

 NetLogo-Inspired  Lua-Powered  Highly Scalable

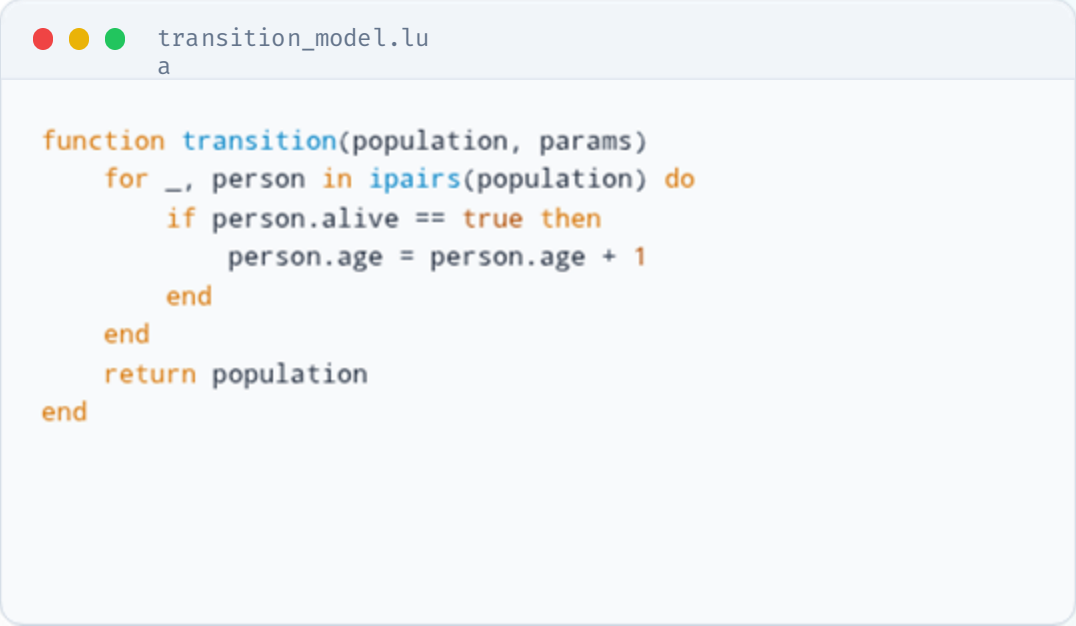
NetLogo-Inspired Philosophy

Core Value	NetLogo Paradigm	Hekate Architecture
Model Logic	Written in NetLogo's custom language	Written in lightweight Lua
Distribution	Single direct application download	Single static execution binary
Usability	Highly accessible syntax, easy to learn	Intuitive modeling interface, fast learning curve
Agility	Rapid model prototyping & deployment	Agile, high-speed model execution loops

Comparison with existing microsim approaches

Core Feature	Existing Platforms (e.g., SimPaths)	Hekate Architecture
User Accessibility	Low, requires Java development skills to adapt, run, re-compile, changes at different parts of the source code required	High, designed for non-programmers to easily implement pathways, transitions, and models
Setup Complexity	Re-compiling within a java code-base setting (intelliJ, Java, etc.)	Download and run executable engine file, operate from within R or Python, as COMPASS
Data Requirements	Data is restricted; training data provided	High degree of transferability due to factory-model design
Modularity	Limited due to dedicated SimPaths model structure and limited ownership of existing model and its further development	Extensions and linkages of underpinning data already possible (e.g., UPRN, Benefit Units and UKMod, mapping across surveys) + further useage in ABMs
Key Strength	Deeply validated, complex underpinning parametrisation of household utility function maximisation and transition models	Rapid transition model prototyping & deployment, modularity, ease of use
Suggestion	n/a	Ship with "basic" configuration of validated demographic and lifecourse processes for further extension with UK Synthetic Population

Engine vs. Model Architecture



```
transition_model.lu
a

function transition(population, params)
  for _, person in ipairs(population) do
    if person.alive == true then
      person.age = person.age + 1
    end
  end
  return population
end
```



Go (Golang) Core Engine

The entire orchestrator and background system is written in Go, there are no dependencies.



Lua Simulation Models

The custom simulation logic, state transitions, and parameters are scripted directly in lightweightc Lua models.



Dynamic Binding

Go streams and binds population datasets instantly into the embedded Lua runtime there is no recompiling

Learning Ramp Simple to Complex

```
function transition(population, params)
  for _, person in ipairs(population) do
    if person.alive == true then
      person.age = person.age + 1
    end
  end
  return population
end
```

```
# MODEL 4: Calculate Mortality Risk and Apply Deaths (FIXED)
- name: mortality_model
  type: lua_model
  priority: 4
  enabled: true
  description: "Calculate mortality risk based on health and demographics"
  parameters:
    script: |
      function transition(population, params)
        for _, person in ipairs(population) do
          local alive = false
          if person.alive == true or person.alive == "true" then
            alive = true
          end

          -- CRITICAL: Skip if already dead
          if not alive then
            goto continue
          end

          local age = tonumber(person.age) or 40
          local health_risk = tonumber(person.health_risk) or 0

          -- CRITICAL: If health_risk is 0, SKIP mortality entirely
          -- This means people with missing health_risk don't die
          if health_risk == 0 then
            person.mortality_risk = "0.0000"
            -- Do NOT apply death, just continue
            goto continue
          end

          -- Only proceed if health_risk is valid (> 0)
          -- Base mortality risk (increases with age)
          local base_risk = 0.001 + (age - 18) * 0.0005

          -- Health risk contribution (linear)
          local health_factor = (health_risk / 100) * 0.15

          -- Additional risk factors
          local smoker_risk = 0
          if person.smoker == "true" or person.smoker == true then
            smoker_risk = 0.03
          end

          local chronic_risk = 0
          if person.chronic_disease == "true" or person.chronic_disease == true then
            chronic_risk = 0.04
          end

          -- Combined mortality risk
          local mortality_risk = base_risk + health_factor + smoker_risk + chronic_risk
```

Documentation and Tutorials

README

Apache-2.0 license



Hekate — Microsimulation Engine

Hekate development branch

Microsimulation Engine

Named after the Greek goddess of crossroads, magic, and transitions, Hekate is a self-contained, dynamically configurable demographic microsimulation system written in Go. **Hekate is a general-purpose microsimulation engine** that can simulate population dynamics including migration, aging, mortality, fertility, and other demographic processes. The name reflects the engine's purpose—guiding populations through the crossroads of life events and demographic transitions.

How Hekate Works: Our Approach

Get started with Hekate through our step-by-step tutorials. Each tutorial builds on the previous one, taking you from beginner to advanced user.

Tutorial	Description	Level
Tutorial 1: Building an Aging Model	Learn the basics by creating a simple aging model. You'll create a population CSV, write your first configuration, run the simulation, and analyze the output.	★ Beginner
Tutorial 2: Understanding YAML and Adding Mortality	Dive deeper into YAML rules. Learn how to add a mortality model with age-specific death probabilities and track population changes.	★★ Intermediate
Tutorial 3: Adding Fertility	Complete the demographic cycle by adding fertility. Learn how to create new individuals (births), assign their characteristics, track population growth, and understand the full demographic cycle of aging, mortality, and fertility.	★★★ Advanced
Tutorial 4: Understanding the Hekate Codebase	A comprehensive deep dive into the Hekate source code. Designed for Go programmers who want to understand how Hekate works under the hood. Includes 7 mini-demos that isolate and explain key concepts like dynamic YAML parsing, Lua embedding, Go-Lua conversion, and the simulation loop.	★★★★★ Expert Programmer
Tutorial 5: Large-Scale Simulations with Streaming Mode	Learn how to use Hekate's streaming mode to simulate populations of millions to billions of people with minimal memory. Includes performance tuning, area-based processing, and best practices for large-scale runs.	★★★★★ Advanced User

Get Help from AI

LLM Prompt Template

Copy, paste, and instantly generate valid YAML and Lua code for your target simulation parameters.



Repository Template (LLM_PROMPT_TEMPLATE.md)

Comprehensive pre-formatted template included directly in the repository.



Paste into ChatGPT, Claude, or any LLM

Seamlessly configure external language models



Describe & Run Code Instantly

Example: "I want a migration model with age-specific rates" to yield operational syntax.



prompt-
migration.yaml

```
# AI-generated Hekate Model Config
models:
  - name: migration_model
    priority: 2
    parameters:
      rates: [ 0.012, 0.045, 0.021 ]
      script: |
        function transition(population, params)
          -- Custom migration steps ...
        end
```

Performance: 1M People · 5 Years

1,000,000

Simulated Agents

25 min

Total Compute Time

~20 MB

Ultra-low Memory Floor

 ACTIVE PIPELINE MODELS

Aging 

Income 

Health Risk 

Mortality 

Hekate: Modern Microsimulation



Models as Data

Clean combination of structural configuration declarative YAML and functional Lua agent actions.



Scales Anywhere

Built to handle computations seamlessly starting from 10 individuals up to 100 million agents.



Built-in Stats

Out-of-the-box analytical structures supporting deep Linear & Logistic dynamic regressions.



Streaming Data

Maintains highly predictable constant memory footprints via sequential state stream design.